Simon Sutoris 7542170

1 Simply Typed Lambda Calculus

$$\dfrac{\dfrac{\dfrac{x : \mathbf{A} \in y : \mathbf{A}, x : \mathbf{A}}{y : \mathbf{A}, x : \mathbf{A} \vdash x : \mathbf{A}}\,\textsc{Ty-Var}}{y : \mathbf{A} \vdash (\lambda x : \mathbf{A}.x) : \mathbf{A} \to \mathbf{A}}\,\textsc{Ty-Abs} \quad \dfrac{y : \mathbf{A} \in y : \mathbf{A}}{y : \mathbf{A} \vdash y : \mathbf{A}}\,\textsc{Ty-Var}}{y : \mathbf{A} \vdash ((\lambda x : \mathbf{A}.x)\, y) : \mathbf{A}}\,\textsc{Ty-App}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{y : \mathbf{C} \in x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B}, w : \mathbf{A}}{x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B}, w : \mathbf{A} \vdash y : \mathbf{C}}\,\textsc{Ty-Var}}{x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B} \vdash (\lambda w : \mathbf{A}.y) : \mathbf{A} \to \mathbf{C}}\,\textsc{Ty-Abs}}{x : \mathbf{B}, y : \mathbf{C} \vdash (\lambda z : \mathbf{B}.\lambda w : \mathbf{A}.y) : \mathbf{B} \to (\mathbf{A} \to \mathbf{C})}\,\textsc{Ty-Abs} \quad \dfrac{x : \mathbf{B} \in x : \mathbf{B}, y : \mathbf{C}}{x : \mathbf{B}, y : \mathbf{C} \vdash x : \mathbf{B}}\,\textsc{Ty-Var}}{x : \mathbf{B}, y : \mathbf{C} \vdash ((\lambda z : \mathbf{B}.\lambda w : \mathbf{A}.y)\, x) : \mathbf{A} \to \mathbf{C}}\,\textsc{Ty-App}$$

There exist no context Γ or type T for the two latter terms t in the exercise. For the first, w is a free variable, and we can't deduce its type without evaluating it: Or in other words: There is no typing rule that lets us determine its type.

For the latter: x must be of a type, that is both a concrete type, and can be evaluated. Both can't be simultaneously true.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{z : \mathbf{A} \in \Gamma, x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{A}}{\Gamma, x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{A} \vdash z : \mathbf{A}}\,\textsc{Ty-Var}}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash (\lambda z : \mathbf{A}.z) : \mathbf{A} \to \mathbf{A}}\,\textsc{Ty-Abs} \quad \dfrac{\lightning}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash (w\, x\, y) : \mathbf{A}}}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash ((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{A}}\,\textsc{Ty-App}}{\Gamma, x : \mathbf{B} \vdash \lambda y : \mathbf{C}.((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{C} \to \mathbf{A}}\,\textsc{Ty-Abs}}{\Gamma \vdash \lambda x : \mathbf{B}.\lambda y : \mathbf{C}.((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{B} \to (\mathbf{C} \to \mathbf{A})}\,\textsc{Ty-Abs}$$

$$\dfrac{\dfrac{\dfrac{\lightning}{x : \tau', y : \mathbf{A} \vdash x : (\tau \to \tau')} \quad \dfrac{x : \tau' \in x : \tau', y : \mathbf{A}}{x : \tau', y : \mathbf{A} \vdash x : \tau'}\,\textsc{Ty-Var}}{x : \tau', y : \mathbf{A} \vdash x\, x : \tau'}\,\textsc{Ty-App}}{x : \tau* \vdash \lambda y : \mathbf{A}.x\, x : \mathbf{A} \to \tau'}\,\textsc{Ty-Abs}$$

2 Extension with Numbers

Theorem 1. *Preservation of Types under Substitution (9.3.8)*

$$\dfrac{\Gamma, x : S \vdash t : T \qquad \Gamma \vdash s : S}{\Gamma \vdash t[s/x] : T}$$

Proof. Case T-Var:
If $t = x$, then $x : S \to x[s/x] = s : S \Rightarrow t$ and $t[s/x]$ have the same type S.
If $t \neq x$, then $t \to t[s/x] : T = t : T$.
Case T-Zero:
$[s/x]t = \texttt{zero} : \texttt{Nat}$
Case T-Abs:
TODO
Case T-App:
TODO
Case T-Succ:
If $t = x$, then $S = \texttt{Nat} \Rightarrow x : \texttt{Nat} \to x[s/x] = s : \texttt{Nat}$.
If $t \neq x$, then $t[s/x] = t : \texttt{Nat}$

□

Theorem 2. *Progress (9.3.5)*
Let t be a closed, well-typed term, that is $\vdash t : T$ for some T, then either t is a value or there is some t' with $t \to t'$.

Proof. T-Var does not apply, because not closed. T-Zero, T-Abs, T-Succ are values by definition.

For T-App, because of the rules axioms $\Gamma \vdash t_1 : T_{11} \to T_{12}$ and $\Gamma \vdash t_2 : T_{11}$, it follows that $t_1\, t_2 : T_{12} \to [x/t_2]t_1 : T_{12}$. □

Theorem 3. *Preservation (9.3.9)*
If $\Gamma \vdash t : T$ and $t \to t'$, then $\Gamma \vdash t' : T$.

Proof. T-Var, T-Zero and T-Abs don't apply, because there are no derivation rules for them.
Case T-Succ:

$$t : \texttt{Nat} \tag{1}$$
$$t' = \texttt{succ}\, t_1 : \texttt{Nat} \tag{2}$$

Because t and t' have the same type `Nat` and $t \to t'$, E-Succ applies.
Case T-App $t_1 \to t_1'$:
TODO
Case T-App $t_1 = v_1 \wedge t_2 \to t_2'$:
TODO
Case T-App $t_1 = (\lambda x : T.t_{12}) \wedge t_2 = v_2$:
TODO

□

3 Environments and free variables

TODO

4 Compilation to untyped lambda calculus

TODO

5 Types and Programming Languages

No questions.